

Frozen Lake: A Q-Learning Solution

<https://github.com/qyau-k/FrozenLake-kog>

Kwame Ofori-Gyau

22424324

Reinforcement Learning Assignment Technical Report

Introduction

Reinforcement Learning is a paradigm of machine learning in which an agent learns to make sequential decisions by interacting with an environment. Unlike supervised learning, the agent receives no labelled examples it learns entirely through trial and error, guided by a reward signal. The fundamental learning loop consists of four elements at every time step: the current state S , the chosen action A , the reward R received, and the resulting next state S' . This tuple (S, A, R, S') is called a *transition* and is the unit of experience from which the agent learns.

Environment Design

State Representation

The 8x8 grid contains 64 cells. Each cell is a state, represented as a single integer from 0 to 63, numbered left-to-right, top-to-bottom ($\text{state} = \text{row} \times 8 + \text{col}$). A single integer was chosen over (row, col) tuples to make the indexing of the q-table simple. Internally, `step()` converts between integer states and coordinates temporarily bound movement to the grid then converts back immediately.

Action Representation

Four discrete actions are available: Left (0), Down (1), Right (2), Up (3). Movement is bounded attempting to move off the grid edge leaves the agent in place. This is enforced using min and max clamping on the row and column coordinates independently.

Reward Structure

Event	Reward	Terminal
Reach Goal (G)	+1.0	Yes
Fall in Hole (H)	0.0	Yes
Any other step (F)	0.0	No

Table 1: Reward structure of the Frozen Lake environment.

The reward signal is sparse positive reward is only given upon reaching the goal. This means the agent must successfully complete the entire path before receiving any learning signal.

Q-Learning Algorithm

Description

Q-Learning is a off-policy temporal difference algorithm. It maintains a Q-table a 64x4 matrix where each entry $Q(s, a)$ represents the expected cumulative discounted reward for taking action a in state s and following the optimal policy thereafter. The Q-table is initialised to zero (no prior knowledge) and updated incrementally after every individual step.

Update Equation

The Q-value update rule applied after every transition is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

Here alpha (learning rate) controls the step size of each update, gamma (discount factor) determines how much future rewards are valued relative to immediate ones, r is the immediate reward, and $\max Q(s', a')$ is the best Q-value available in the next state. The bracketed term is the temporal difference error the gap between the current estimate and the updated target. When an episode terminates (done=True), there is no next state and the target simplifies to just r.

Exploration Strategy

Epsilon-greedy exploration is used throughout training. With probability epsilon the agent selects a random action (exploration); with probability 1-epsilon it selects the action with the highest Q-value (exploitation). Epsilon decays multiplicatively after each episode:

```
epsilon <- max(epsilon_min, epsilon * (1 - decay_rate))
```

Multiplicative decay was selected over additive decay following experimentation. Additive decay with rate 0.001 reduced epsilon to its minimum after only 990 episodes far too early given that purely random exploration finds the goal only 0.49% of the time. Multiplicative decay with rate 0.0001 keeps meaningful exploration active for approximately 46,000 of the 50,000 training episodes.

Training Methodology

Training runs for 50,000 episodes. Each episode begins by resetting the agent to state 0. The agent then takes steps

each step selects an action via epsilon-greedy, calls step() on the environment to obtain (S', R, done), applies the Q-update, and advances to S'. Episodes are capped at 1,000 steps to prevent indefinite wandering. Epsilon decays once per episode. After training, the Q-table and statistics are saved to disk for evaluation and plotting.

Hyperparameter	Value	Rationale
Episodes	50,000	Needed given sparse reward difficulty
Max steps/episode	1,000	Gives random walks time to reach the goal
Learning rate (alpha)	0.8	Fast convergence on deterministic problem
Discount factor (gamma)	0.95	Values rewards ~20 steps ahead
Epsilon start	1.0	Fully random no prior knowledge
Epsilon minimum	0.01	Retains 1% exploration throughout
Epsilon decay	0.0001 (mult.)	Reaches minimum at ~46,000 episodes

Table 2: Final hyperparameter configuration.

Experimental Results

Metric	Value
Training success rate (last 1,000 eps)	97.5%

Evaluation success rate (100 episodes, epsilon=0)	100.0%
Average reward (evaluation)	1.000
Successful runs (evaluation)	100 / 100
Failures (evaluation)	0 / 100
Episode at which 50% success rate reached	Approx. 12,800
Episode at which 95% success rate reached	Approx. 35,860

Table 3: Evaluation results after 50,000 training episodes.

The agent achieved a perfect 100% success rate on evaluation with no exploration. Training success rate during the final 1,000 episodes was 97.5% the 2.5% failure rate is attributable to the residual epsilon=0.01 random exploration still active during training. The learning curve shows a steady sigmoid-shaped improvement: slow initial progress as the Q-table accumulates sparse goal signals, accelerating improvement as more states gain positive Q-values, and plateau convergence once the optimal policy is established.

Learned Policy

The policy extracted from the trained Q-table is shown below. For each non-terminal state, the arrow indicates the action with the highest Q-value. H and G mark terminal states.

```

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ → → → →
↓ ↓ ↓ ↓ → → ↑ H ↓ ↓ ↓ ↓
→ → ↑ H ↓ ↓ ↓ ↓ → → ↑ H
↓ ↓ ↓ ↓ → → ↑ H ↓ ↓ → ↓
↑ H H → → ↓ H ↓ ↑ H → ↑
H ↓ ↑ ← ← H → → → G

```

Challenges Encountered

The first challenge was trying to figure out the movement then I did some research and found out about the (x,y) strategy of movement and implemented that. Then it realized as well that I had to create boundaries for the grid so that the agent stays within the 0 to 63 integers.

The first two runs gave 0% success rate with a correct implementation. The agent learned nothing. The problem was the epsilon decay strategy. Additive decay of 0.001 killed exploration after just 990 episodes. By then the agent had barely figured out the goal so the Q-table was still nearly all zeros. Then for the remaining 9000+ episodes it just kept repeating the same empty policy.

To discover the cause, I had to check how often a purely random agent finds the goal and for 1000 steps the random success rate was 0.49. The maze is hard to navigate randomly. The hole column at rows 2 to 4 and the cluster at rows 5 to 6 block most random paths. The agent needed thousands of lucky episodes before the Q-table had enough signal to start exploiting.

Conclusion

This project demonstrates a complete Q-Learning solution for the Frozen Lake problem implemented entirely from scratch. The agent successfully learned to navigate a sparse-reward 8x8 grid environment, achieving 100% success on evaluation after 50,000 training episodes. All components environment, agent, training loop, and evaluation were built in pure Python using only NumPy.

The key technical insight was the importance of exploration duration. Additive epsilon decay is a common default but proved fatal on this problem because the reward signal is too sparse for the agent to encounter it frequently enough during a short exploration phase. Multiplicative decay gave the agent the extended exploration window it needed to build a complete Q-table.